



Slide 1
CS 170
Java Programming 1
All About Scope
 Duration: 00:01:05
 Advance mode: Auto

Notes:

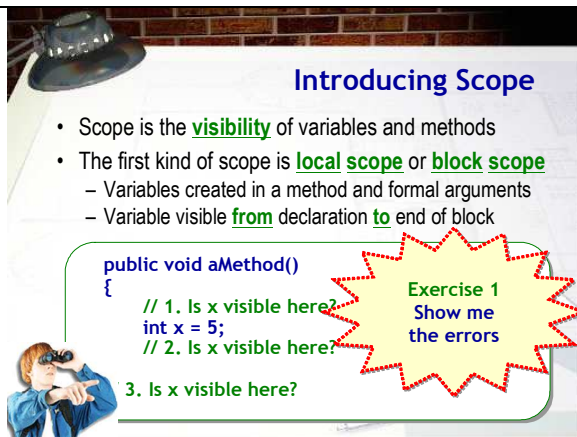
Hi Folks. Welcome to the CS 170, Java Programming 1 lecture, All About Scope.

Most of us are pretty attuned to the sound of our names. If you are at a party, and you hear someone call out "Sam!", you'll probably look around and see if someone is trying to get your attention. (Assuming your name is Sam, of course.)

On the other hand, if you hear the name "Sam" in a movie, or on TV, you're unlikely to do the same thing, because the context makes it clear that you're not the Sam under discussion.

Variable names work in a similar way; the same name can mean different things, depending upon the context. As I'm sure you remember, a variable is simply a named memory location that stores a value. You'll also remember that instance variables are different than local variables. One of the things differentiating instance variables and local variables is the scope of variable names.

That's what we'll look at in this lecture.



Slide 2
Introducing Scope
 Duration: 00:01:32
 Advance mode: Auto

Notes:

Scope is word that simply means "visibility", or ...

...where is the name of a variable visible?

In this section, we'll look at two kinds of scope: local scope and class scope.

Variables created inside a method, or inside a method "block" have local scope. (A method block is the part of a method enclosed in braces; normally a method block is the body of a loop or of a selection statement.) Formal arguments to a method also have local scope.

To explore this concept, create an empty class in Eclipse, named AClass. Inside AClass, add the method named aMethod, shown here. At the lines marked 1, 2, and 3, use System.out.println() to print the value stored in the variable x. Save your code and for Exercise 1, shoot a screenshot of the Eclipse window. You'll have red underscores on the lines where x is not visible or, as we say, "out of scope". Start a new section in your IC document and paste the picture as Exercise 1.

When you're done, comment out the illegal lines and save your work to make sure the remaining uses of x are legal.

• Identifier names must be unique

```
public void aMethod()
{
    int x = 3;           // OK, only x in scope
    double x = 2.5;      // ERROR, x already used
}
```

• Can use same variable name in **different** scopes

```
public void aMethod() { int x = 3;           // OK }
public void bMethod() { double x = 2.5;      // OK }
```

Slide 3

Local Scope

Duration: 00:01:37

Advance mode: Auto

Notes:

Two variables in the same scope may not have the same name. Make these changes to aMethod and save your file. Eclipse will show you a squiggly red line. Hover over the line to see the Eclipse error message. For Exercise 2, shoot me a screenshot of that error.

Each local method is a different local scope, and variables in different local scopes may have the same name. Add a new method named bMethod() and move the second definition of x down into it. Save your work, and, as you'll see, the error disappears.

The variable x defined inside bMethod() is different than the variable x defined inside aMethod(). The body of aMethod() is a different local scope than the body of bMethod().

Variables in different local scopes are invisible to each other, and may not be accessed from another local scope. You could not access the int variable x (to print it or use it in a calculation, for instance), from inside the body of bMethod.

Formal parameters to a method also have local scope for the entire method. That means that you cannot have a local variable that has the same name as one of the method's formal parameters. To see that this is true, add a formal parameter named x to aMethod, save your work, and notice the error message. Then, remove the formal parameter before going on.

• The second type of scope is **class scope**

- Fields (instance vars) & methods have class scope
- Visible inside any method (even before declaration)
- Not visible to **fields** defined **previous** to declaration

```
public class AClass
{
    public void aMethod() { x = 5; // OK }
    public void bMethod() { x = 7; // OK }
    private int x = 3; // can't use PUBLIC_FIELD
    public int PUBLIC_FIELD = 5;
}
```

Slide 4

Class Scope

Duration: 00:01:49

Advance mode: Auto

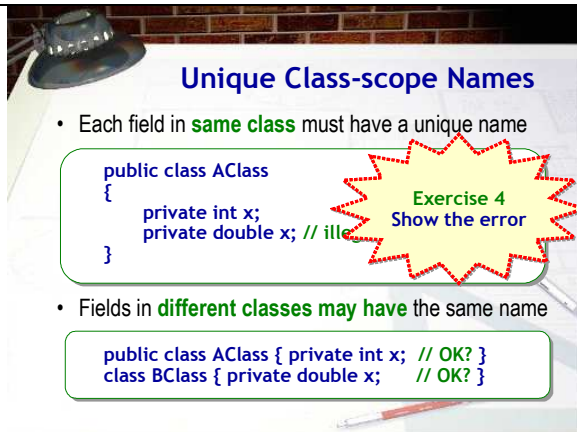
Notes:

Class scope refers to the scope rules that apply to instance variables (or fields) and to methods. The scope of an instance variable is the class it is defined in. To see how this works, modify AClass like this and save the file. Notice that I've removed the local variables inside aMethod() and bMethod(). Instead, I've added an instance variable named x and a second public instance variable named PUBLIC_FIELD. Scope is also affected by the access modifiers public, private, and protected. We'll ignore that complication for right now and just look at basic visibility.

Notice that variables with class scope, like x, are visible inside both aMethod and inside bMethod, even though the instance variable x was defined after the definition for the two methods. However, an instance variable like PUBLIC_FIELD is not visible to other instance variables that are declared before it. Let's see how that works.

- Change the line that initializes x, so that instead of assigning the literal value 3, use the PUBLIC_FIELD to initialize the variable. For Exercise 3, show me the error message.
- Now, move the definition of PUBLIC_FIELD so that it appears before the definition for the instance variable x. Save your file and you notice that the error message

disappears. Now PUBLIC_FIELD is visible to x.



Unique Class-scope Names

- Each field in **same class** must have a unique name

```
public class AClass
{
    private int x;
    private double x; // illegal
}
```

Exercise 4
Show the error

- Fields in **different classes** may have the same name

```
public class AClass { private int x; // OK? }
class BClass { private double x; // OK? }
```

Slide 5

Unique Class-scope Names

Duration: 00:01:28

Advance mode: Auto

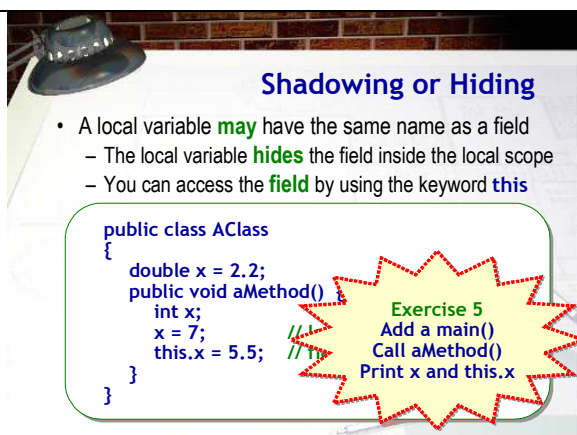
Notes:

With class scope, you cannot have two instance variables with the same name, in the same class, just like you cannot have two local variables with the same name in the same method. Each field must have a unique name.

Let's see if that's true. Add a second instance variable named x to AClass. Shoot me a screenshot of the error and paste it into you're IC document as Exercise 4.

Now, since you can't have two instance variables with the same name in the same class, what happens when two instance variables, in two different classes, have the same name, like this? Try it out. In the same file as AClass, add a second class named BClass. (Don't put the word public in front of the class, since you can have only one public class per file.). Move the declaration for the double x down to BClass. What happens?

Well, as you can see, this is fine. Both class A, and class B are different class scopes, so the names don't clash with each other at all. That means, just like with local variables, methods in class A cannot access the instance variable x in class B, and vice-versa.



Shadowing or Hiding

- A local variable **may** have the same name as a field
 - The local variable **hides** the field inside the local scope
 - You can access the **field** by using the keyword **this**

```
public class AClass
{
    double x = 2.2;
    public void aMethod()
    {
        int x;
        x = 7;
        this.x = 5.5; // OK
    }
}
```

Exercise 5
Add a main()
Call aMethod()
Print x and this.x

Slide 6

Shadowing or Hiding

Duration: 00:01:05

Advance mode: Auto

Notes:

Instance variables and local variables exist in different scopes, so a local variable may have the same name as an instance variable. When that happens, the most local variable name takes precedence. Here's an example.

Since the most local identifier takes precedence, assigning 7 to x local variable x "wins out" over the field x. If the method needs to access the field x, it can do so by prefixing the word this as in the line that assigns 5.5 to this.x.

For the last exercise in this lecture, go ahead and add a main() method to AClass. In the main() method, create an AClass object and call aMethod(). Inside aMethod() add two lines of code to print x and to print this.x. Run it and show me a screenshot for Exercise 5.